TRIBHUVAN UNIVERSITY

PATAN MULTIPLE CAMPUS

PATANDHOKA, LALITPUR

BIT 252 ARTIFICIAL INTELLIGENCE

SUBMITTED BY:

NAME: SURESH DAHAL
CLASS: BIT II/II
ROLL NO.:23
DATE: _____

SUBMITTED TO:

RAKESH KUMAR BACHCHAN

_____
CHECKED BY

1. TITLE: SIMPLE REFLEX AGENT FOR PARTICLE PICKING PROBLEM

THEORY:

A Simple Reflex Agent is an AI agent that makes decisions based solely on the current percept, without considering the history of previous actions or states. In the Particle Picking Problem, the agent operates in an environment with two rooms — A and B. Each room can be either Dirty or Clean.

The agent:

1. Checks if the current room is dirty.
2. If dirty → cleans it.
3. Otherwise → moves to the other room.

This behavior follows the Condition–Action rule:

IF status == Dirty → Clean ELSE → Move to another room.

ALGORITHM:

1. **Start**
2. Sense the **current room** (A or B).
3. Check the **status** of the current room.
4. **If the room is Dirty:**
   a. Perform **Clean** action.
5. **Else (room is Clean):**
   a. Move to the **other room**.
6. Repeat steps **2–5** until **both rooms are Clean**.
7. **Stop**.

EXAMPLE:

Initial Environment

Room A = dirty

Room B = dirty Agent

starts in Room A

Goal:
Clean both rooms in minimum steps.

PROGRAM:

print (""" name:

suresh dahal

```
roll_no: 23
lab_no:1
    """)
def reflex_agent(room, status):     if status ==
'Dirty':         return 'Clean'     return 'Move to B' if
room == 'A' else 'Move to A'
# Initialize environment roomA,
roomB = 'Dirty', 'Dirty'
current_room, step = 'A', 1
max_steps = 4
print(f"Start: A={roomA},
B={roomB},
Agent={current_room}")

while (roomA == 'Dirty' or roomB == 'Dirty') and step <= max_steps:
    status = roomA if current_room == 'A' else roomB
action = reflex_agent(current_room, status)     if
action == 'Clean':         if current_room == 'A':
            roomA = 'Clean'
else:             roomB =
'Clean'     else:
        current_room = 'B' if current_room == 'A' else 'A'

    print(f"Step {step}: Percept=({current_room},{status}) → Action={action} → "
f"State[A={roomA}, B={roomB}, Agent={current_room}]")
    step += 1

print("\nSUCCESS: All clean!" if roomA == 'Clean' and roomB == 'Clean'
else "\nHALT: Max steps reached")
```

OUTPUT:

```
In [5]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:1

Start: A=Dirty, B=Dirty, Agent=A
Step 1: Percept=(A,Dirty) → Action=Clean → State[A=Clean, B=Dirty, Agent=A]
Step 2: Percept=(B,Clean) → Action=Move to B → State[A=Clean, B=Dirty, Agent=B]
Step 3: Percept=(B,Dirty) → Action=Clean → State[A=Clean, B=Clean, Agent=B]

SUCCESS: All clean!
```

2. TITLE: UTILITY-BASED AGENT FOR SIMPLE WUMPUS WORLD

THEORY:

A Utility-Based Agent is an intelligent agent that makes decisions not only to achieve goals but also to maximize its utility. Unlike a Simple Reflex Agent, which just reacts to conditions, a Utility-Based Agent evaluates how good or bad an action's outcome is — using a utility function.

In the Wumpus World, the agent moves through a grid environment that may contain:

- Gold → desirable (positive utility)
- Wumpus (monster) → dangerous (negative utility)
- Pit → deadly (negative utility)
- Empty cell → neutral

The agent's goal is to maximize total utility by choosing the safest, most rewarding moves.

ALGORITHM:

1. Start

2. For each possible action:

   Calculate utility of resulting cell

3. Choose action with highest utility

4. Move to that cell

5. Repeat until goal reached or no safe moves

6. End

EXAMPLE:

Consider a 2×2 Wumpus World:

A1→Start, Breeze	A2→Pit
B1→Safe	B2→Gold, Breeze

The agent starts at A1, moves below to B1 and then to B2.

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
lab_no:2
    """)
# Define the world world
= {
    'A1': {'breeze': True, 'pit': False, 'gold': False},
    'A2': {'breeze': False, 'pit': True, 'gold': False},
    'B1': {'breeze': False, 'pit': False, 'gold': False},
    'B2': {'breeze': True, 'pit': False, 'gold': True}
}

# Possible moves moves
= {
    'A1': ['A2', 'B1'],
    'A2': ['A1', 'B2'],
    'B1': ['A1', 'B2'],
    'B2': ['A2', 'B1']
}

def perceive(location):     cell =
world[location]     return
cell['breeze'], cell['gold']
```

```
def utility_based_agent(start):
    current = start     actions = []
print(f"Agent starts at {current}")

    for step in range(3):  # small world → few steps         breeze, glitter =
perceive(current)         print(f"\nStep {step+1}: At {current} |
Breeze={breeze}, Gold={glitter}")
                if
glitter:
            actions.append("Grab gold")
print("Action: Grab gold")
print("\nSUCCESS: Gold collected!")
            break

        # Decision rules
if current == 'A1':
if breeze:

                print("Percept of breeze, A2 is risky, moving to B1 instead")
next_move = 'B1'
            else:
                next_move = 'A2'

        elif current == 'B1':             print("No
breeze → Safe to explore B2 ")
next_move = 'B2'

else:
break
```

```
        actions.append(f"Move to {next_move}")
print(f"Action: Move to {next_move}")        current
= next_move

        # Pit check         if
world[current]['pit']:            print("Fell
into pit! Game over.")            return
actions, False

    return actions, world[current]['gold']

# Run the agent actions, success =
utility_based_agent('A1') print("\nActions taken:", ',
'.join(actions)) print("Result:", "SUCCESS" if
success else "FAIL")
```

OUTPUT:

```
In [19]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:2

Agent starts at A1

Step 1: At A1 | Breeze=True, Gold=False
Percept of breeze, A2 is risky, moving to B1 instead
Action: Move to B1

Step 2: At B1 | Breeze=False, Gold=False
No breeze → Safe to explore B2
Action: Move to B2

Step 3: At B2 | Breeze=True, Gold=True
Action: Grab gold

SUCCESS: Gold collected!

Actions taken: Move to B1, Move to B2, Grab gold
Result: SUCCESS
```

3. TITLE: GOAL-BASED AGENT FOR SIMPLE WUMPUS WORLD

THEORY:

A Goal-Based Agent is an intelligent agent that chooses actions to achieve a specific goal, rather than just reacting or maximizing utility. It has knowledge of the environment, a set of goals, and uses search or reasoning to decide which actions will lead to the goal. In Simple Wumpus World, the goal is usually to find the gold while avoiding the Wumpus and pits. The agent considers possible actions and moves only if they help it reach the goal safely.

EXAMPLE:

Consider a 2×2 Wumpus World:

Cell	Description
A1	Safe
A2	Wumpus (-100)
B1	Gold (+100)
B2	Pit (-50)

Goal: Reach the Gold (B1) safely from A1.

The agent checks its possible moves and chooses the path that leads to the goal without danger.

ALGORITHM:

1. Start at A1
2. Set Goal = "Find Gold"
3. Check safe neighboring cells of A1
4. If a neighbor is safe → move there
5. If cell contains Gold → stop
6. Else continue checking next safe cell

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
lab_no:3
    """)

# Define the world world
= {
    'A1': 'Safe',
    'A2': 'Wumpus',
    'B1': 'Gold',
    'B2': 'Pit'
}

# Neighboring moves moves
= {
    'A1': ['A2', 'B1'],
```

```
    'A2': ['A1', 'B2'],
    'B1': ['A1', 'B2'],
    'B2': ['A2', 'B1']
}

goal = 'Gold'

def goal_based_agent(current):
    print(f"Agent starts at {current}\n")
step = 1     while True:
        print(f"Step {step}: Current Position = {current}")
neighbors = moves[current]         print(f"  Possible
moves: {neighbors}")

        # Check neighbors for goal
found = False         for move in
neighbors:
            print(f"  Evaluating {move} -> {world[move]}")
if world[move] == goal:
                print(f"Goal found at {move}, moving there!")
current = move                  found = True                break

        if found:
            print(f"\nSUCCESS: Gold collected at {current}!")
            break
else:
            print("No goal in neighboring cells. Agent halts.")
break         step += 1

# Run agent goal_based_agent('A1')
```

OUTPUT:

```
In [25]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:3

Agent starts at A1

Step 1: Current Position = A1
  Possible moves: ['A2', 'B1']
  Evaluating A2 -> Wumpus
  Evaluating B1 -> Gold
Goal found at B1, moving there!

SUCCESS: Gold collected at B1!
```

4. TITLE: MODEL-BASED AGENT IN A SIMPLE WUMPUS WORLD

THEORY:

A Model-Based Agent maintains an internal model of the environment to make better decisions even when it doesn't perceive the whole world directly. In the Wumpus World, the agent uses percepts (like Breeze or Stench) to update its internal belief about which cells are safe or dangerous. Unlike simple reflex agents, a model-based agent remembers what it has seen and uses that memory to plan safe movements and reach the goal (Gold).

ALGORITHM:

1. Start at the initial cell.
2. Check if current cell has Gold → grab it.
3. Mark current cell as visited.
4. Check adjacent cells (UP, DOWN, LEFT, RIGHT).
5. Move to a neighbor if it's safe and not visited.
6. Repeat steps 2–5 until Gold is found or no safe moves remain.
7. Output the plan.

EXAMPLE:

Consider a 4×4 Wumpus World:

' ' = Safe, 'W' = Wumpus, 'P' = Pit, 'G' = Gold, 'B' = Breeze, 'S' = Stench

	0	1	2	3

0			P	
1		W		
2				G
3		P		

The agent starts at (3,0) and uses its model (safe/visited knowledge) to find a path to the Gold.

PROGRAM:

```
print ("""     name:

suresh dahal

roll_no: 23

lab_no:4

    """)

# Wumpus World: ' ' = Safe, 'W' = Wumpus, 'P' = Pit, 'G' = Gold, 'B' = Breeze, 'S' = Stench

world = [

    [' ', ' ', 'P', ' '],

    [' ', 'W', ' ', ' '],

    [' ', ' ', ' ', 'G'],

    [' ', 'P', ' ', ' ']

]

percepts = [

    [' ', ' ', 'B', ' '],

    [' ', 'S', ' ', ' '],

    [' ', ' ', ' ', ' '],

    [' ', 'B', ' ', ' ']

]


start, plan, visited = (3, 0), [], set()
```

```
def safe(r, c): return percepts[r][c] not in ('B', 'S') def neighbors(r, c):     for dr,
dc, move in [(-1,0,'UP'), (1,0,'DOWN'), (0,-1,'LEFT'), (0,1,'RIGHT')]:
        nr, nc = r+dr, c+dc         if 0
<= nr < 4 and 0 <= nc < 4:
            yield nr, nc, move


def dfs(r, c, path):     if (r, c) in
visited: return False
visited.add((r, c))

    if world[r][c] == 'G':
        plan.extend(path + ['GRAB']); return True
for nr, nc, m in neighbors(r, c):
        if safe(nr, nc) and dfs(nr, nc, path + [m]): return True
return False


dfs(*start, [])
print("Plan to reach gold:\n" + "\n".join(f" - {step}" for step in plan) if plan else "No safe path
found.")
```

OUTPUT:

```
In [26]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:4

Plan to reach gold:
 - UP
 - RIGHT
 - RIGHT
 - UP
 - RIGHT
 - DOWN
 - GRAB
```

5. TITLE: SOLVING 8-PUZZLE USING BFS ALGORITHM

THEORY:

The 8-Puzzle Problem consists of a 3×3 grid with eight numbered tiles and one empty space (blank). The objective is to move the tiles (by sliding them into the blank space) to reach a goal state from a given initial state. The Breadth-First Search (BFS) algorithm is an uninformed search strategy that explores all possible states level by level. It guarantees the shortest path to the goal (in terms of the number of moves), but it can be slow and memory-intensive for large puzzles.

ALGORITHM:

1. Start with initial state, mark visited, add to queue.
2. While queue not empty:
    - Remove first state.
    - If goal → return path.
    - Find blank (0) and possible moves.
    - For each move: swap tiles → new state.
        - If not visited → mark visited, add to queue.
3. If goal not found → no solution.
4. Output steps to reach goal.

EXAMPLE:

Initial state:

1 2 3

4 0 6

7 5 8

Goal state:

1 2 3

4 5 6 7 8 0

Here 0 represents the blank space.

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
   lab_no:5
   """)
from collections import deque

goal = "123456780" start
= "123406758"

moves = {0:[1,3], 1:[0,2,4], 2:[1,5], 3:[0,4,6],
        4:[1,3,5,7], 5:[2,4,8], 6:[3,7], 7:[4,6,8], 8:[5,7]}

def bfs(start):
   q, visited = deque([(start, [])]), {start}
while q:
       state, path = q.popleft()
if state == goal: return path
i = state.index("0")        for m
in moves[i]:            new_state =
list(state)            new_state[i],
new_state[m] = new_state[m],
new_state[i]            ns =
"".join(new_state)            if ns
not in visited:
visited.add(ns)
                q.append((ns, path+[ns]))
return None
```

```
result = bfs(start)


if result:
    print("Steps to reach goal:")
for step in result:        for i in
range(0,9,3):
print(step[i:i+3])        print()
else:     print("No solution
found.")
```

OUTPUT:

```
In [27]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:5

Steps to reach goal:
123
456
708

123
456
780
```

6. TITLE: SOLVING 8-PUZZLE USING DFS ALGORITHM

THEORY:

The 8-Puzzle Problem is a 3×3 grid with eight numbered tiles and one empty space (0). The goal is to move tiles by sliding them into the blank space to reach a desired goal state. The Depth-First Search (DFS) algorithm is an uninformed search method that explores as far as possible along each branch before backtracking. Unlike BFS, which finds the shortest path, DFS may go deep in one direction first and does not guarantee the shortest solution, but it uses less memory.

ALGORITHM:

1. Start with initial state, mark visited.
2. If state = goal → return path.

3. Find blank (0) and possible moves.
4. For each move: swap tiles → new state.
   - If not visited → recursively call DFS (increase depth).
   - Stop if depth limit reached.
5. Backtrack if no moves left.
6. Output steps if goal found, else report no solution.

EXAMPLE:

Initial State

1 2 3

4 0 6

7 5 8

Goal State

1 2 3

4 5 6 7 8 0

Here 0 represents the blank space.

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
   lab_no:6
   """)
goal = "123456780" start = "123406758"
moves = {0:[1,3], 1:[0,2,4], 2:[1,5], 3:[0,4,6],
        4:[1,3,5,7], 5:[2,4,8], 6:[3,7], 7:[4,6,8], 8:[5,7]}

visited = set() solution
= [] def dfs(state,
path, depth=0,
limit=15):
```

```
    if depth > limit:  # limit depth to prevent infinite recursion
return False     if state == goal:         solution.extend(path)
return True     visited.add(state)     i = state.index("0")     for
m in moves[i]:
        new_state = list(state)         new_state[i], new_state[m] =
new_state[m], new_state[i]         ns = "".join(new_state)
if ns not in visited:             if dfs(ns, path+[ns], depth+1,
limit):
                return True
return       False       if
dfs(start, [start]):
    print("Steps to reach goal:")
for i in range(0,9,3):
        print(solution[-1][i:i+3]) else:
    print("No solution found (depth limit reached).")
```

OUTPUT:

```
In [40]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:6

Steps to reach goal:
123
456
780
```

7. TITLE: SOLVING 8-PUZZLE USING ITERATIVE DEEPENING DFS

THEORY:

The Iterative Deepening Depth-First Search (IDDFS) combines the space efficiency of DFS with the completeness of BFS. It repeatedly applies Depth-Limited Search (DLS) starting from depth 0 and gradually increasing the depth limit — until the goal is found.

- DFS may go too deep and miss shorter solutions.
- BFS finds the shortest path but uses a lot of memory.
- IDDFS balances both: it finds the optimal solution (like BFS) while using less memory (like DFS).

It’s ideal for large state spaces like the 8-Puzzle, where the search tree can grow very fast.

ALGORITHM:

1. Set depth limit = 0.
2. Repeat until goal found or max depth reached:
   a. Perform Depth-Limited Search (DLS) from start.
   b. Mark visited states.
   c. If goal reached → return path.
   d. Increase depth limit by 1.
3. In DLS:
   a. If state = goal → return path.
   b. Find blank (0) and possible moves.
   c. For each unvisited move → swap tiles → recurse with depth+1.
4. If goal not found → report no solution.
5. Output steps to reach goal.

EXAMPLE:

Initial State

1 2 3

4 5 6

7 8 0

Goal State

1 2 3

4 0 6

7 5 8

PROGRAM

```
 Print(“”””
 lab_no:7
   """)


goal = "123456780"
```

```
start = "123406758"

moves = {0:[1,3], 1:[0,2,4], 2:[1,5], 3:[0,4,6],
        4:[1,3,5,7], 5:[2,4,8], 6:[3,7], 7:[4,6,8], 8:[5,7]}

def dls(state, path, depth, limit, visited):
if depth > limit:        return None     if
state == goal:        return path
visited.add(state)     i = state.index("0")
for m in moves[i]:
        new_state = list(state)        new_state[i], new_state[m] =
new_state[m], new_state[i]        ns = "".join(new_state)
if ns not in visited:
            result = dls(ns, path + [ns], depth + 1, limit, visited)
if result:                return result     return None

def iddfs(start, max_depth=15):
for limit in range(max_depth + 1):
        visited = set()        result = dls(start,
[start], 0, limit, visited)        if result:
return result     return None


result = iddfs(start)


if result:
    print("Steps to reach goal:")
for s in result:        for i in
range(0,9,3):
```

```
        print(s[i:i+3])
print() else:
    print("No solution found within depth limit.")
```

OUTPUT:

```
In [41]: %runfile D:/Suresh/code.py -

    name: suresh dahal
    roll_no: 23
    lab_no:7

Steps to reach goal:
123
406
758

123
456
708

123
456
780
```

8. TITLE: PATH PROBLEM SOLUTION USING BEST FIRST SEARCH

THEORY:

Best First Search (BFS*) is a heuristic search algorithm that tries to reach the goal node as quickly as possible. It uses a heuristic value h(n), which tells how close each node is to the goal. At every step, it chooses the node with the lowest heuristic value and expands it next.

- It is faster than normal BFS because it uses heuristics to guide the search.
- But it doesn't always find the shortest path, only a quick one.

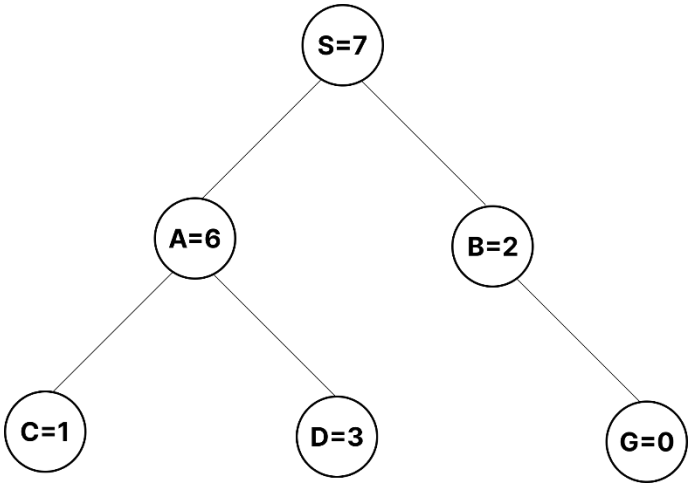
ALGORITHM:

1. Start at the initial node, mark visited set empty.
2. Add start node to queue with heuristic and path.
3. While queue not empty:
   a. Sort queue by heuristic (lowest first) and pop first node.
   b. If node = goal → return path.
   c. Mark node as visited.
   d. For each neighbor:
      i. If not visited → add to queue with heuristic and updated path.
4. If queue empties without goal → report no path found.

5. Output path if found.

EXAMPLE:

Let's take a small graph where the numbers show heuristic values (h) for each node:



Goal: Find a path from S (start) to G (goal) using Best First Search.

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
lab_no:8
    """)

# Define graph and heuristic graph
= {
    'S': ['A', 'B'],
    'A': ['C', 'D'],
    'B': ['G'],
    'C': [],
    'D': [],
    'G': []
}
```

```
h = {'S':7, 'A':6, 'B':2, 'C':1, 'D':3, 'G':0}

def best_first_search(start, goal):
    visited = set()     queue = [(h[start], start, [start])]  #
(heuristic, node, path)

    while queue:
        # sort queue by heuristic and pop the first element
queue.sort(key=lambda x: x[0])
        _, node, path = queue.pop(0)

        if node == goal:
return path

        visited.add(node)

        for neighbor in graph[node]:
if neighbor not in visited:
                queue.append((h[neighbor], neighbor, path + [neighbor]))

    return None

# Run the search result =
best_first_search('S', 'G')

if result:
    print("Path found:", " → ".join(result)) else:
    print("No path found.")
```

OUTPUT:

```
In [42]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:8

Path found: S → B → G
```

9. TITLE: PATH PROBLEM SOLUTION USING A* SEARCH

THEORY:

A* is an informed search algorithm used to find the shortest path between a start and a goal. It uses both actual and estimated costs to make intelligent decisions:

$$f(n) = g(n) + h(n)$$

Where:

- g(n) → cost from start to node n
- h(n) → estimated (heuristic) cost from node n to goal
- f(n) → total estimated cost

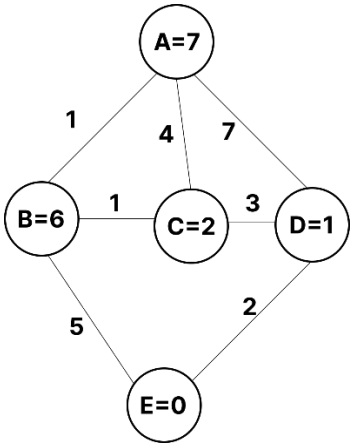
A* expands the node with the lowest f(n) making it both efficient and optimal.

ALGORITHM:

1. Start at node, g(start)=0, visited={}
2. Add start to PQ with f=g+h, path=[start]
3. While PQ not empty:
   a. Pop node with lowest f
   b. If node=goal → return path, cost
   c. Mark node visited
   d. For each neighbor:
      i. new_g = g + cost
      ii. f = new_g + h
      iii. If neighbor not visited or new_g<g → update g, add to PQ
4. If goal not found → report no path
5. Output path and cost

EXAMPLE:

Consider the following graph:



PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
lab_no:9
    """)

from queue import PriorityQueue

# Graph as adjacency list: (neighbor, cost) graph
= {
    'A': [('B', 1), ('C', 4), ('D', 7)],
    'B': [('A', 1), ('C', 1), ('E', 5)],
    'C': [('A', 4), ('B', 1), ('D', 3)],
    'D': [('A', 7), ('C', 3), ('E', 2)],
    'E': []
}

# Heuristic (estimated cost to goal) h
= {'A': 7, 'B': 6, 'C': 2, 'D': 1, 'E': 0}
```

```
def a_star(start, goal):
pq = PriorityQueue()
pq.put((0, start, [start]))
g = {start: 0}     visited =
set()

   while not pq.empty():
f, node, path = pq.get()
if node == goal:
return path, g[node]
visited.add(node)

       for neighbor, cost in graph[node]:
           new_g = g[node] + cost            f = new_g + h[neighbor]
if neighbor not in visited or new_g < g.get(neighbor, float('inf')):
               g[neighbor] = new_g
pq.put((f, neighbor, path + [neighbor]))     return
None, float('inf')

# Run A* Search
path, cost = a_star('A', 'E')
print("Shortest Path:", path) print("Total
Cost:", cost)
```

OUTPUT:

```
In [44]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:9

Shortest Path: ['A', 'C', 'D', 'E']
Total Cost: 9
```

10. TITLE: BLOCK WORLD USING HILL CLIMBING SEARCH

THEORY:

Hill Climbing is a search algorithm that always moves towards a better (higher value) state. It looks at all the neighboring states and chooses the one that seems closest to the goal.

- It keeps improving the current state step by step.
- Stops when no better move is found (goal or a dead end).
- It is like climbing a hill until the top is reached.

ALGORITHM:

1. Start with initial state.
2. Compute heuristic (blocks in correct position).
3. Repeat:
   a. Generate all neighbor states (swap any two blocks).
   b. Select neighbor with highest heuristic.
   c. If neighbor heuristic ≤ current → stop.
   d. Else → move to neighbor, record step.
4. Output steps.
5. If last state = goal → goal reached; else → local maximum.

EXAMPLE:

- **Goal:** Stack blocks **A on B on C** (top-down: A→B→C)
- **Initial state:** All blocks separate: **[A, B, C]** on table

Heuristic (h): Number of blocks in correct position in stack (top-down)

- Example states and h values:

State	h

[A,B,C] 0

[B,C,A] 1

[C,B,A] 2
Goal: [A,B,C] 3

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
   lab_no:10
   """)

initial_state = ['C', 'A', 'B'] goal_state
= ['A', 'B', 'C']

def heuristic(state):
   # number of blocks in correct position from top     return sum([1 if
state[i] == goal_state[i] else 0 for i in range(len(state))])

def hill_climbing(state):
   steps = [state[:]]
while True:
      neighbors = []
      # Generate neighbors by swapping any two blocks
for i in range(len(state)):            for j in range(i+1,
len(state)):

            new_state = state[:]              new_state[i],
new_state[j] = new_state[j], new_state[i]
neighbors.append(new_state)        # Select neighbor with max
```

```
heuristic        next_state = max(neighbors, key=heuristic)
if heuristic(next_state) <= heuristic(state):
            # no better neighbor → stop
break        state = next_state
steps.append(state[:])     return
steps


# Run Hill Climbing path = hill_climbing(initial_state) print("Steps to reach goal
(Hill Climbing):") for step in path:     print(step) print("\nGoal reached!" if path[-1]
== goal_state else "\nStopped at local maximum.")
```

OUTPUT:

```
In [45]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:10

Steps to reach goal (Hill Climbing):
['C', 'A', 'B']
['A', 'C', 'B']
['A', 'B', 'C']

Goal reached!
```

11. TITLE: AND GATE USING HEBBIAN NEURAL NETWORK

THEORY:

Hebbian Learning Rule is one of the earliest learning rules in neural networks. This rule says the weight between two neurons becomes stronger if both are active at the same time.

For an AND gate, we have two inputs (X_1, X_2) and one output (Y). The neuron should output 1 only when both inputs are 1.

Hebbian Rule Formula:

$$\Delta w_i = n \times x_i \times y$$

Where:

- Δw_i: Change in weight
- n: Learning rate (usually 1 for simplicity)
- x_i: Input value
- y: Output

ALGORITHM:

1. Initialize weights w1, w2, bias b = 0, learning rate n=1.
2. For each input pattern (x1, x2) with target y:
   a. Update weights: w1 += n*x1*y, w2 += n*x2*y
   b. Update bias: b += n*y
3. After training, test each input:
   a. Compute output = 1 if (w1*x1 + w2*x2 + b) > threshold else 0
4. Output results for all inputs.

EXAMPLE:

X_1	X_2	Target (Y)
0	0	0
0	1	0
1	0	0
1	1	1

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
lab_no:11
    """)
```

```
# Input patterns and target outputs for AND gate
inputs = [[0,0],[0,1],[1,0],[1,1]] targets =
[0,0,0,1]

# Initialize weights and bias
w1 = w2 = b = 0 learning_rate
= 1

# Hebbian learning process for
i in range(len(inputs)):
    x1, x2 = inputs[i]     y = targets[i]     w1 += learning_rate * x1 *
y     w2 += learning_rate * x2 * y     b += learning_rate * y
print(f"After input {inputs[i]} → w1={w1}, w2={w2}, b={b}")

# Testing phase print("\nTesting
AND gate:") for x1, x2 in
inputs:
    output = 1 if (w1*x1 + w2*x2 + b) > 1.5 else 0
print(f"Input: ({x1}, {x2}) → Output: {output}")
```

OUTPUT:

```
In [46]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:11

After input [0, 0] → w1=0, w2=0, b=0
After input [0, 1] → w1=0, w2=0, b=0
After input [1, 0] → w1=0, w2=0, b=0
After input [1, 1] → w1=1, w2=1, b=1

Testing AND gate:
Input: (0, 0) → Output: 0
Input: (0, 1) → Output: 1
Input: (1, 0) → Output: 1
Input: (1, 1) → Output: 1
```

12. TITLE: AND GATE USING PERCEPTRON NEURAL NETWORK

THEORY:

A Perceptron is the simplest type of artificial neural network.
It takes inputs, multiplies them by weights, adds a bias, and passes the result through an activation function to produce an output.

It learns using the Perceptron Learning Rule, which adjusts weights based on the error between the actual and desired outputs.

Learning Rule:

$$w_i = w_i + \eta(t - y)x_i$$
$$b = b + \eta(t - y)$$

Where:

- w_i: weight
- η: learning rate
- t: target output
- y: actual output
- x_i: input

ALGORITHM:

1. Initialize weights w1, w2, bias b = 0, learning rate η.
2. For each epoch:
   a. For each input (x1, x2) with target t:
      i. Compute output y = 1 if (w1*x1 + w2*x2 + b) > 0 else 0
      ii. Compute error = t - y
      iii. Update weights: w1 += η*error*x1, w2 += η*error*x2
      iv. Update bias: b += η*error
3. After training, test each input:
   a. Output y_pred = 1 if (w1*x1 + w2*x2 + b) > 0 else 0
4. Display results for all inputs.

EXAMPLE:

X_1	X_2	Target (Y)
0	0	0
0	1	0
1	0	0
1	1	1

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
lab_no:12
    """)
# Input and target output for AND gate
X = [[0,0],[0,1],[1,0],[1,1]]
Y = [0,0,0,1]

# Initialize weights and bias
w1 = w2 = b = 0 eta = 1  #
learning rate

# Training using Perceptron rule for epoch in
range(5):  # small number of iterations
print(f"Epoch {epoch+1}")     for i in range(4):
x1, x2 = X[i]        y_in = w1*x1 + w2*x2 + b
y = 1 if y_in > 0 else 0        error = Y[i] - y
```

```
        w1 += eta * error * x1          w2 += eta * error * x2          b += eta * error
print(f"Input: {X[i]}, Target: {Y[i]}, Output: {y}, New w1={w1}, w2={w2}, b={b}")
    print()

# Testing print("Testing the trained
Perceptron:") for x1, x2 in X:
    y_pred = 1 if (w1*x1 + w2*x2 + b) > 0 else 0
print(f"Input: ({x1}, {x2}) → Output: {y_pred}")
```

OUTPUT:

```
    name: suresh dahal
    roll_no: 23
    lab_no:12

Epoch 1
Input: [0, 0], Target: 0, Output: 0, New w1=0, w2=0, b=0
Input: [0, 1], Target: 0, Output: 0, New w1=0, w2=0, b=0
Input: [1, 0], Target: 0, Output: 0, New w1=0, w2=0, b=0
Input: [1, 1], Target: 1, Output: 0, New w1=1, w2=1, b=1

Epoch 2
Input: [0, 0], Target: 0, Output: 1, New w1=1, w2=1, b=0
Input: [0, 1], Target: 0, Output: 1, New w1=1, w2=0, b=-1
Input: [1, 0], Target: 0, Output: 0, New w1=1, w2=0, b=-1
Input: [1, 1], Target: 1, Output: 0, New w1=2, w2=1, b=0

Epoch 3
Input: [0, 0], Target: 0, Output: 0, New w1=2, w2=1, b=0
Input: [0, 1], Target: 0, Output: 1, New w1=2, w2=0, b=-1
Input: [1, 0], Target: 0, Output: 1, New w1=1, w2=0, b=-2
Input: [1, 1], Target: 1, Output: 0, New w1=2, w2=1, b=-1

Epoch 4
Input: [0, 0], Target: 0, Output: 0, New w1=2, w2=1, b=-1
Input: [0, 1], Target: 0, Output: 0, New w1=2, w2=1, b=-1
Input: [1, 0], Target: 0, Output: 1, New w1=1, w2=1, b=-2
Input: [1, 1], Target: 1, Output: 0, New w1=2, w2=2, b=-1

Epoch 5
Input: [0, 0], Target: 0, Output: 0, New w1=2, w2=2, b=-1
Input: [0, 1], Target: 0, Output: 1, New w1=2, w2=1, b=-2
Input: [1, 0], Target: 0, Output: 0, New w1=2, w2=1, b=-2
Input: [1, 1], Target: 1, Output: 1, New w1=2, w2=1, b=-2

Testing the trained Perceptron:
Input: (0, 0) → Output: 0
Input: (0, 1) → Output: 0
Input: (1, 0) → Output: 0
Input: (1, 1) → Output: 1
```

13. TITLE: BACKPROPAGATION NEURAL NETWORK TRAINING ALGORITHM

THEORY:

The Backpropagation Algorithm is a supervised learning method used to train multilayer neural networks.
It works by adjusting the network's weights to minimize the error between the predicted output and the actual target.

The process has two main phases:

1. Forward Pass:
    - Input values move through the network (from input → hidden → output layers).
    - Each neuron calculates its output using weight and activation functions.

2. Backward Pass (Backpropagation):
    - The network calculates the error at the output layer.
    - This error is sent backward to adjust the weights of neurons (using gradient descent).
    - The idea is to make small changes that reduce the overall error.

This process repeats many cycles (called epochs) until the network's predictions are accurate.

ALGORITHM:

1. Initialize weights w1, w2, bias randomly, set learning rate.
2. For each epoch:
   a. For each input (x1, x2) with target t:
      i. Forward pass: out = sigmoid(x1*w1 + x2*w2 + bias)
      ii. Compute error = t - out
      iii. Backward pass: delta = error * sigmoid_derivative(out)
      iv. Update weights: w1 += lr*delta*x1, w2 += lr*delta*x2
      v. Update bias: bias += lr*delta
3. Repeat until error is minimized or max epochs reached.
4. After training, test each input:
   a. Output = sigmoid(x1*w1 + x2*w2 + bias)
5. Display results.

EXAMPLE:

Let's train a small network to learn the OR gate:

Input 1	Input 2	Target
0	0	0
0	1	1
1	0	1
1	1	1

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
lab_no:13
    """)

import math import
random

# Sigmoid activation function and its derivative def
sigmoid(x):
    return 1 / (1 + math.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Training data for OR gate inputs
= [[0,0], [0,1], [1,0], [1,1]] targets
= [0, 1, 1, 1]

# Initialize random weights
w1 = random.random() w2
= random.random() bias =
random.random()
learning_rate = 0.1

# Training process for epoch
in range(5000):
```

```
total_error = 0     for i in
range(len(inputs)):         x1,
x2 = inputs[i]         target =
targets[i]

        # Forward pass         net =
x1*w1 + x2*w2 + bias         out
= sigmoid(net)

        # Calculate error
error = target - out
total_error += error**2

        # Backward pass (weight updates)
delta = error * sigmoid_derivative(out)
w1 += learning_rate * delta * x1         w2 +=
learning_rate * delta * x2         bias +=
learning_rate * delta

    if epoch % 1000 == 0:
        print(f"Epoch {epoch}: Error = {total_error:.4f}")

# Test the trained network for
x1, x2 in inputs:
    out = sigmoid(x1*w1 + x2*w2 + bias)
print(f"Input: {x1}, {x2} → Output: {out:.3f}")
```

OUTPUT:

```
In [48]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:13

Epoch 0: Error = 0.6942
Epoch 1000: Error = 0.0770
Epoch 2000: Error = 0.0340
Epoch 3000: Error = 0.0211
Epoch 4000: Error = 0.0151
Input: 0, 0 → Output: 0.081
Input: 0, 1 → Output: 0.950
Input: 1, 0 → Output: 0.949
Input: 1, 1 → Output: 1.000
```

14. TITLE: GENETIC ALGORITHM IMPLEMENTATION

THEORY:

A Genetic Algorithm (GA) is an optimization and search method inspired by the process of natural selection — like how living things evolve over time.
It uses the ideas of selection, crossover, and mutation to find the best solution to a problem.

Steps of GA:

1. Initialization: Start with a random population of solutions.
2. Selection: Choose the best solutions (parents) based on how good they are (fitness).
3. Crossover: Mix two parents to create new children (offspring).
4. Mutation: Randomly change a small part of some solutions to keep variety.
5. Repeat: Continue for many generations until a good solution is found.

ALGORITHM:

1. Initialize random population of solutions.

2. Evaluate fitness of each solution.

3. Repeat for many generations:

   a. Select parents based on fitness.

   b. Perform crossover to create offspring.

   c. Apply mutation to offspring.

   d. Form new population from offspring.

4. After final generation, select best solution.

5. Output best solution and its fitness.

EXAMPLE:

Find the value of x(0–31) that maximizes the function

$$f(x) = x^2$$

Each x represented as a 5-bit binary string (e.g., 10110 = 22).

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
lab_no:14
    """)

import random

# Fitness function
def fitness(x):
return x ** 2

# Convert binary string to integer
def binary_to_int(b):     return
int(b, 2)

# Generate random population def
generate_population(size):
return [''.join(random.choice('01')
```

```
for _ in range(5)) for _ in
range(size)]

# Selection (Tournament method) def
select(population):
    selected = random.sample(population, 2)     return
max(selected, key=lambda x: fitness(binary_to_int(x)))

# Crossover def
crossover(p1, p2):
    point = random.randint(1, 4)     return p1[:point] +
p2[point:], p2[:point] + p1[point:]

# Mutation def
mutate(chromosome, rate=0.1):
    return ".join(         bit if random.random() > rate
else str(1 - int(bit))         for bit in chromosome
    )

# Genetic Algorithm main function def
genetic_algorithm(generations=10, population_size=6):
    population = generate_population(population_size)
print("Initial Population:", population)

    for gen in range(generations):
        new_population = []         while
len(new_population) < population_size:
            parent1 = select(population)
parent2 = select(population)            child1,
```

```
child2 = crossover(parent1, parent2)
new_population.extend([mutate(child1),
mutate(child2)])        population =
new_population[:population_size]


    best = max(population, key=lambda x: fitness(binary_to_int(x)))
    print(f"Gen {gen + 1}: Best = {best} ({binary_to_int(best)}), Fitness =
{fitness(binary_to_int(best))}")


  best_final = max(population, key=lambda x: fitness(binary_to_int(x)))
  print(f"\nBest Solution: {best_final} (x = {binary_to_int(best_final)}) → f(x) =
{fitness(binary_to_int(best_final))}")
```

Run genetic_algorithm()

OUTPUT:

```
In [49]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:14

Initial Population: ['00011', '10100', '10100', '01001', '00101', '10101']
Gen 1: Best = 11100 (28), Fitness = 784
Gen 2: Best = 11101 (29), Fitness = 841
Gen 3: Best = 11111 (31), Fitness = 961
Gen 4: Best = 11101 (29), Fitness = 841
Gen 5: Best = 11101 (29), Fitness = 841
Gen 6: Best = 11101 (29), Fitness = 841
Gen 7: Best = 11110 (30), Fitness = 900
Gen 8: Best = 11110 (30), Fitness = 900
Gen 9: Best = 11110 (30), Fitness = 900
Gen 10: Best = 11111 (31), Fitness = 961

Best Solution: 11111 (x = 31) → f(x) = 961
```

15. TITLE: SPAM DETECTION USING NAIVE BAYES CLASSIFIER

THEORY:

The Naive Bayes Classifier is a simple probabilistic algorithm used for classification problems — especially in text processing like spam detection.

It is based on Bayes' theorem, which calculates the probability that a message belongs to a certain class (e.g., *Spam* or *Ham*) given the words it contains.

The word *"Naive"* means the algorithm assumes that all words in a message are independent (which isn't always true, but works well in practice).

Steps:

1. Collect and label messages as *Spam* or *Ham (Not Spam)*.
2. Count how often each word appears in both classes.
3. Use probabilities to classify new messages as spam or not.

ALGORITHM:

1. Take all training messages and mark them as Spam or Ham.

2. Count how many times each word appears in Spam and in Ham messages.

3. For a new message:

   a. Start with P(Spam) = spam messages / total messages

      P(Ham) = ham messages / total messages

   b. For each word in the message:

      - Multiply P(Spam) by (word count in Spam + 1) / (total Spam words + vocabulary size)

      - Multiply P(Ham) by (word count in Ham + 1) / (total Ham words + vocabulary size)

4. Compare P(Spam) and P(Ham):

   - If P(Spam) > P(Ham), classify as Spam

   - Else, classify as Ham

5. Repeat for all new messages.

EXAMPLE:

Let's classify messages as Spam or Ham using simple training data.

Message	Label
buy cheap now	Spam
limited offer	Spam
win big prize	Spam
hello friend	Ham

let’s meet tomorrow	Ham
see you soon	Ham

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
lab_no:15
    """)

# Training Data train_data
= [
    ("buy cheap now", "spam"),
    ("limited offer", "spam"),
    ("win big prize", "spam"),
    ("hello friend", "ham"),
    ("let’s meet tomorrow", "ham"),
    ("see you soon", "ham")
]

# Step 1: Count words in spam and ham
spam_words = {} ham_words = {}
spam_count = ham_count = 0

for message, label in train_data:
words = message.split()     if
label == "spam":
```

```
spam_count += 1         for word
in words:
            spam_words[word] = spam_words.get(word, 0) + 1
    else:
        ham_count += 1
for word in words:
            ham_words[word] = ham_words.get(word, 0) + 1

# Step 2: Probability calculation function def
predict(message):     words = message.split()
p_spam = spam_count / (spam_count + ham_count)
p_ham = ham_count / (spam_count + ham_count)

    # Multiply probabilities of each word (add log to avoid underflow)
for word in words:
        p_spam *= (spam_words.get(word, 0) + 1) / (sum(spam_words.values()) +
len(spam_words))
        p_ham *= (ham_words.get(word, 0) + 1) / (sum(ham_words.values()) +
len(ham_words))

    return "SPAM" if p_spam > p_ham else "HAM"

# Step 3: Test test_messages
= [
    "buy offer now",
    "hello offer",
    "meet friend tomorrow",
    "cheap prize"
]
```

```
print("SPAM DETECTION RESULTS:\n")

for msg in test_messages:

    print(f"Message: '{msg}' → {predict(msg)}")
```

OUTPUT:

```
In [50]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:15

SPAM DETECTION RESULTS:

Message: 'buy offer now' → SPAM
Message: 'hello offer' → HAM
Message: 'meet friend tomorrow' → HAM
Message: 'cheap prize' → SPAM
```

16. TITLE: PARSE TREE GENERATION OF A SENTENCE

THEORY:

A parse tree (or syntax tree) shows the grammatical structure of a sentence according to a given grammar. It is built using context-free grammar (CFG), a set of production rules that define how words and phrases combine. For example:

Sentence → Noun Phrase + Verb Phrase
Noun Phrase → Determiner + Noun
Verb Phrase → Verb + Noun Phrase

A parser takes a sentence and applies these rules to show how the sentence can be derived from the grammar step by step forming a tree structure.

ALGORITHM:

1. Define a grammar using rules for Sentence, Noun Phrase, Verb Phrase, etc.

2. Input a sentence and convert it to lowercase words.

3. Use a parser (like nltk.ChartParser) with the defined grammar.

4. Apply parser to the sentence:

   a. Match words with grammar rules.

   b. Build tree structure showing how sentence is derived.

5. If a valid parse is found:

   - Display the parse tree.

- Draw the tree visually.

6. If no parse is found:

- Show message: "Sentence cannot be parsed with this grammar."

EXAMPLE:

Input string:

The fox was brown and big.

Det → Determiner (The)

N → Noun (fox)

V → Verb (was)

Adj → Adjective (brown, big)

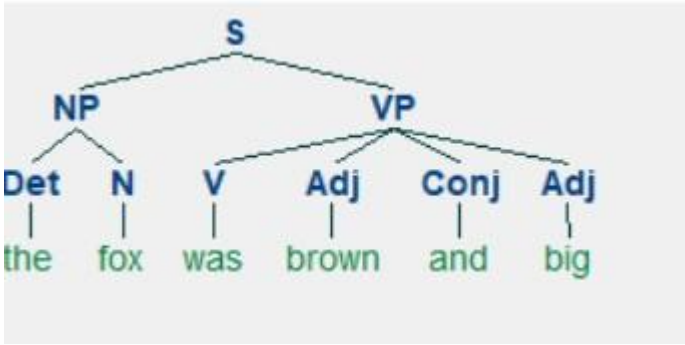
Conj → Conjunction (and)

PROGRAM:

```
print ("""     name:
suresh dahal
roll_no: 23
lab_no:16
    """)

import nltk from nltk
import CFG # Define
a simple grammar
grammar =
CFG.fromstring("""
S -> NP VP
  NP -> Det N | Det N PP | N
  VP -> V NP | VP PP | V | V Adj | V Adj Conj Adj
  PP -> P NP
  Det -> 'the' | 'a'
```

```
  N -> 'cat' | 'dog' | 'mat' | 'telescope' | 'fox'
  V -> 'saw' | 'sat' | 'barked' | 'was'
  P -> 'on' | 'with' | 'under'
  Adj -> 'brown' | 'big'
  Conj -> 'and'
""")

# Create parser parser =
nltk.ChartParser(grammar)

# Take input from user sentence = input("Enter a
sentence: ").lower().split()

# Parse the sentence
print("\nParse Trees:\n") found =
False for tree in
parser.parse(sentence):
    print(tree)
tree.draw()     found
= True

if not found:
    print("Sorry, the given sentence cannot be parsed with the defined grammar.")
```

OUTPUT:




```
In [5]: %runfile D:/Suresh/code.py --wdir

    name: suresh dahal
    roll_no: 23
    lab_no:16

Enter a sentence: the fox was brown and big

Parse Trees:

(S
  (NP (Det the) (N fox))
  (VP (V was) (Adj brown) (Conj and) (Adj big)))
```